

实验 1 复杂地形穿越

在本实验中你将学到：

- 怎样设计基本的游戏；
- 怎样应用地形知识构建特定于游戏的世界；
- 怎样向游戏中添加对象以提供交互性；
- 怎样测试和调整完成的游戏。

在本实验中，你将消化迄今为止所学的知识，并使用它们构建你的第一个 Unity 游戏。你首先将了解游戏的基本设计元素。接着，将构建游戏发生的世界。然后，将添加一些交互性对象，以使玩家能够玩游戏。最后，将开始玩游戏，并执行任何必要的调整以改进体验。

提示：

完成的项目

一定要遵循本实验中的指导来构建完整的游戏项目。如果你不知所措，可以在本实验配套资源中找到该游戏的完成版本。如果需要帮助或灵感，可以看一看它。

1.1 设计

游戏开发的设计部分用于提前规划游戏的所有主要特性和组件。可以把它视作是布置一份蓝图，使得实际的构造过程要顺畅得多。在创建游戏时，有许多时间通常都花在完成设计上。由于你将在本实验中创建的游戏相当基本，因此设计阶段将完成得很快。在创建这款游戏时，你需要重点关注规划的 3 个领域：理念、规则和需求。

1.1.1 理念

这款游戏背后的思想很简单。从一个区域的一边开始，并迅速行进到另一边。在前进的道路上将会出现山丘、树木和各种障碍物。你的目标是看看可以多快地完成它。之所以为你的第一款游戏选择这种游戏理念，是因为它突出显示了你迄今为止处理过的所有领域。此外，由于你正在学习在 Unity 中编写脚本，因此不能添加非常精巧的交互。将来的游戏将更复杂。

1.1.2 规则

每个游戏都必须具有一组规则。规则服务于两个目的：第一，它们告诉你玩家实际上将怎样玩游戏；第二，因为软件是一个许可的过程（参见下面的“注意：许可的过程”），规则指定了可供玩家征服挑战所采用的动作。用于 **Amazing Racer** 游戏的规则如下。

- 没有获胜或失败的条件，只有完成的条件。当玩家进入完成区域时，就完成了游戏。
- 玩家总是从相同的地点复活，完成区域也总是位于相同的地点。
- 将会出现水障。无论何时玩家陷入水障中，都会把该玩家移回复活点（spawn point）。
- 游戏的目标是尝试尽可能获得最快的时间。这是一条隐含的规则，并且没有明确地构建到游戏中。作为替代，将在游戏中构建一些线索，暗示玩家这就是目标。其思想是：玩家将基于提供给他们信号凭直觉期望更快的时间。

➤

注意：

许可的过程

在创建游戏时，始终要记住软件是一个许可的过程。这意味着除非明确指定允许什么，否则它将对玩家不可用。例如，如果玩家希望爬树，但是你没有创建任何方式让玩家爬树，那么

就不允许执行那个动作。如果你没有给玩家跳跃的能力，他们就不能跳跃。你希望玩家能够做的一切事情都必须构建到游戏中。记住：不能假定任何动作，必须为一切都做好规划！

注意：

术语

本实验中使用了下面一些新术语。

- 复活：复活是一个过程，玩家或实体通过它进入游戏。
- 复活点：复活点是玩家或实体复活的位置，游戏中可以有一个或多个复活点，它们可以是静止的，或者是四处移动的。
- 条件：条件是一种触发器形式。获胜条件是使玩家赢得游戏的事件（比如积累足够的点数）；失败条件是使玩家输掉游戏的事件（比如丢失所有的单击点数）。
- 游戏控制器：游戏控制器规定了游戏的规则和流程。它负责知道何时赢得或输掉游戏（或者只是游戏结束了）。可以把任何对象指定为游戏控制器，只要它总是在场景中即可。通常，把一个空对象或者 **Main Camera** 指定为游戏控制器。

1.1.3 需求

设计过程中的另一个重要步骤是确定游戏将需要哪些资源。一般来讲，游戏开发团队由多人组成。其中一些人将从事设计，其他人则负责编写程序或创建艺术。团队的每位成员在开发过程中的每个步骤当中都需要做某件事情来提高效率。如果每一个人都等待某件事情完成才能开始工作，那么将会出现许多开始和停止。作为替代，你可以提前确定资源，以便在事物需要之前就可以创建它们。下面列出了 **Amazing Racer** 游戏的所有需求。

- 一块矩形区域的地形。地形需要足够大，以展示一场具有挑战性的比赛。地形中应该具有一些障碍，以及指定的复活点和终点，如图 1.1 所示。
- 用于地形的纹理和环境效果。它们是在 **Unity** 标准资源中提供的。
- 一个复活点对象、一个完成区域对象和一个水障对象。将在 **Unity** 中生成它们。
- 一个角色控制器。这是由 **Unity** 标准资源提供的。
- 一个图形用户界面（GUI）。将在本实验配套资源中为你提供它。
- 一个游戏控制器。将在 **Unity** 中创建它。

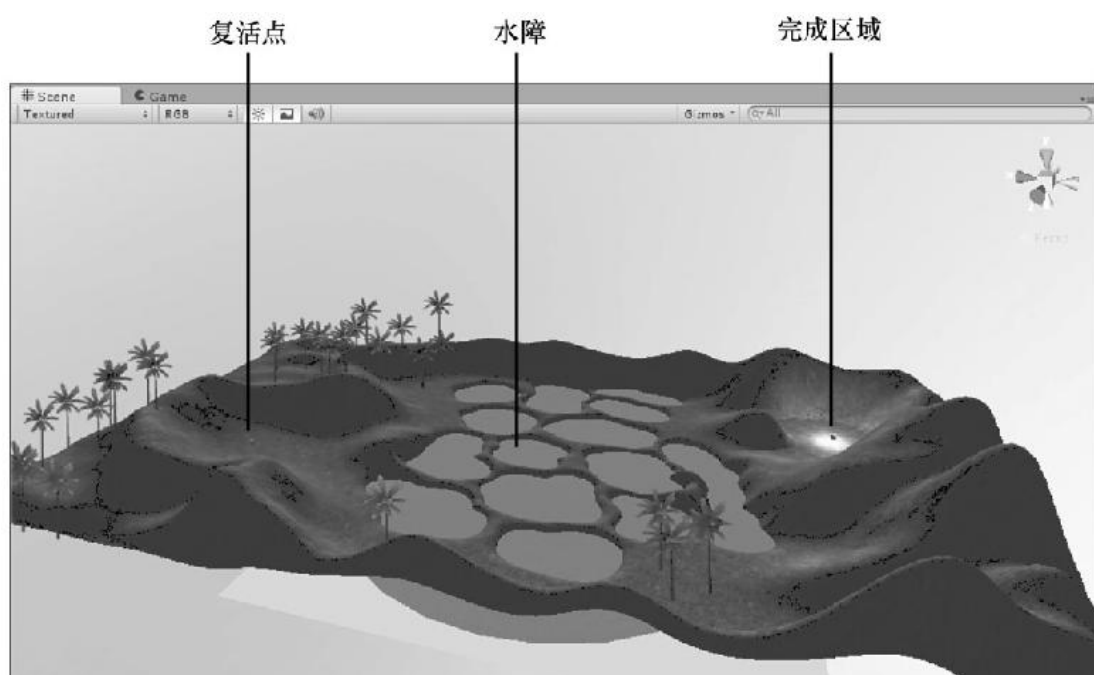


图 1.1 AmazingRacer 游戏的一般地形布局

1.2 创建游戏世界

既然你已经在纸上写下了游戏的基本思想，现在就可以开始构建它。可以在许多位置开始构建一款游戏。对于这个项目，可以从游戏世界开始。由于这是一款线性赛车游戏，游戏世界的长度将大于它的宽度（或者它的宽度将大于它的长度，这依赖于你如何查看它）。将使用许多 Unity 标准资源快速创建该游戏。

1.2.1 雕刻游戏世界

可以用许多方式创建这个地形。每一个人都会有不同构思。为了简化这个过程，将为你提供一幅高度图。它用于确保每一个人在本实验中都将具有相同的经历。要雕刻地形，可以遵循下面这些步骤。

（1）在名为 **Amazing Racer** 的文件夹中创建一个新项目，并向该项目中添加一个地形。

（2）把该地形的分辨率设置为 200（宽）×100（长）×100（高）（在 **Terrain Settings** 的 **Resolution** 区域中设置它）。

（3）在用于本实验配套资源中定位文件 **terrain.raw**。导入该文件作为地形的高度图（通过在 **Terrain Settings** 的 **Heightmap** 区域中单击 **Import Raw** 命令）。

（4）在资源下面创建一个 **Scenes** 文件夹，并把当前场景另存为 **Main**。

现在应该雕刻地形，以匹配本实验中的游戏世界。可以自由地执行微小的调整和修改，以使其具有你喜欢的样子。

警告：

构建你自己的地形

在本实验中，你将基于所提供的高度图构建一款游戏。高度图已经为你准备好了，以便你可以迅速体验游戏开发的过程。不过，你也可以选择构建自定义的游戏世界，以使这款游戏独一无二，并且是属于你自己的一款游戏。不过，如果你这样做，就要当心提供给你的一些坐标和旋转角度可能不匹配。如果你想构建自己的游戏世界，就要注意打算放置对象的位置，并把它们相应地放在游戏世界中。

1.2.2 添加环境

此时，你可以开始纹理化地形，并向其中添加一些环境效果。你需要进入 **Assets Store** 搜索并下载资源包 **Low-Poly Simple Nature Pack** 并导入资源包，详情请参见教材 P161-P162 第 2 步-第 5 步的操作过程。在 **Project** 窗口中 **Assets** → **low poly package** 目录中有

➤ Skybox

在 **Project** 窗口中 **Assets** → **low poly package** → **Standard Assets** 目录中有

➤ Environment

➤ Characters

➤ Effects

- 你现在可以有一点自由，按你所喜欢的方式装饰游戏世界。下面的建议是指导原则，可以以自己的喜好来安排事情。参见教材 P164 第 9 步-第 19 步的相关操作。
- 向场景中添加一种定向灯光，旋转它以适应你的偏好。
- 纹理化地形。示例项目使用以下纹理：**Grass (Hill)**或 **layer_Grass ER** 用于平坦部分，**Cliff (Layered Rock)**或 **NewLayer 7** 用于陡峭部分，**Grass&Rock** 或 **NewLayer 1** 用于它们之间的区域。

- 向场景中添加一个天空盒。
- 向地形中添加一些树木。应该稀疏地放置树木，并且主要放在平坦的表面上。
- 向场景中添加一些基本的水（在 Project 视图中从 Assets → low poly package → Standard Assets\Water (Basic)文件夹中拖动 WaterBasicDaytime）。把水放在(88, 29, 49)处，并把它缩放成(50, 1, 50)。

现在应该就准备好了地形，并且准备在它上面行走了。一定要花相当多的时间进行纹理化，以确保具有良好的混合和逼真的外观。在示例项目中还有一些额外的事物没有展示出来，但是你可能想添加它们，包括以下几点。

- 雾。
- 水障周围的青草。这可能会把它们遮掩一点，并且增加了难度。
- 用于定向灯光的光晕，用于模拟太阳。需要旋转定向光源，以匹配天空盒的太阳图像（如果具有天空盒的话）。

1.2.3 角色控制器

在开发的这个阶段，你将希望向场景中添加一个角色控制器。

（1）单击 Assets → low poly package → Standard Assets → Characters，导入标准的角色控制器。

（2）从 Assets → low poly package → Standard Assets → Characters → FirstPersonCharacter → Prefabs 文件夹中把 FPSController 控制器资源拖入场景中。

（3）把 FPSController 控制器（它将被命名为 Player 并设置为蓝色）放置在(160, 32, 64)处。在 y 轴上把控制器旋转 260°，使之面朝正确的方向。

一旦角色控制器位于场景中并且定位了它，就可以播放场景。一定要四处看看，寻找需要修补或平滑的任何区域。要注意边界。寻找你能够逃离游戏世界的任何区域，将需要抬高这些位置，使得玩家将不能脱离地图。在这个阶段，一般将修正地形的任何基本的问题。

提示：

脱离游戏世界

一般来讲，游戏关卡将具有一些水或者其他一些障碍，以阻止玩家退出已开发的区域。如果游戏利用重力，玩家可能会脱离游戏世界的边缘。你总是希望创建某种方式，阻止玩家到达某个他们不应到达的区域。这个游戏项目使用高高的护堤使玩家保持在游戏区域内。在用于本实验配套资源中提供的高度图有意给出了几个位置，玩家可以从中爬出。看看你是否能够找到并校正它们。

1.3 游戏化

现在，你拥有了一个可以玩游戏的游戏世界。你可以四处逛逛，并在一定程度上体验这个游戏世界。现在，所遗漏的部分正是游戏本身。目前，你所拥有的只是一个玩具，只可以玩一玩。你想要的是一款游戏，可它只是一个具有一定规则和一定目标的玩具。把某件东西转变成游戏的过程称为游戏化（gamification），这就是本节将要做的事情。如果你遵循了前面的步骤，那么游戏项目现在看上去应该如图 1.2 所示。下面几个步骤是添加一些游戏控制对象以便进行交互，为那些对象应用游戏脚本，并把它们彼此联系起来。

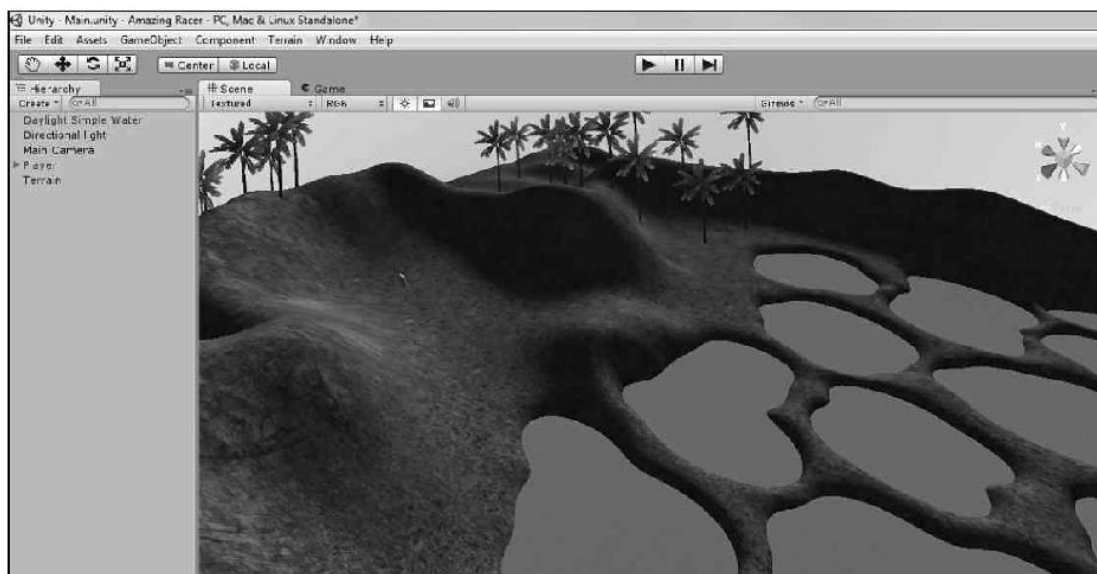


图 1.2 Amazing Racer 游戏的当前状态

注意：

脚本

脚本是为游戏对象定义行为的代码段。你正在学习在 **Unity** 中编写脚本。不过，要创建交互式游戏，脚本是必须的。考虑到这一点，我们为你提供了制作这款游戏所需的脚本。我们努力使脚本尽可能小，以便你可以理解这个项目的大部分内容。可以自由地在文本编辑器中打开脚本，并阅读它们所做的事情。

1.3.1 添加游戏控制对象

如在 1.1.3 节中所定义的，需要 4 个特定的游戏控制对象。第一个对象是复活点。这将是一个简单的游戏对象，只用于告诉游戏在哪里复活玩家。要创建复活点，可以遵循下面这些步骤。

(1) 向场景中添加一个空的游戏对象（单击 **GameObject > Create Empty** 命令），并把它定位于(160, 32, 64)处。

(2) 在 **Hierarchy** 视图中把空对象重命名为 **SpawnPoint**。

接下来，创建水障检测器。它应该是一个位于水下的简单平面，并具有一个触发碰撞器，用于检测玩家何时落入水中。要创建检测器，可以遵循下面这些步骤。

(1) 向场景中添加一个平面（单击 **GameObject > Create Other > Plane** 命令），把它定位于(86, 27, 51)处，并把该平面缩放为(10, 1, 10)。

(2) 在 **Hierarchy** 视图中把平面重命名为 **WaterHazardDetector**。

(3) 在 **Inspector** 视图中选中 **Mesh Collider** 组件上的 **Is Trigger** 复选框，如图 1.3 所示。

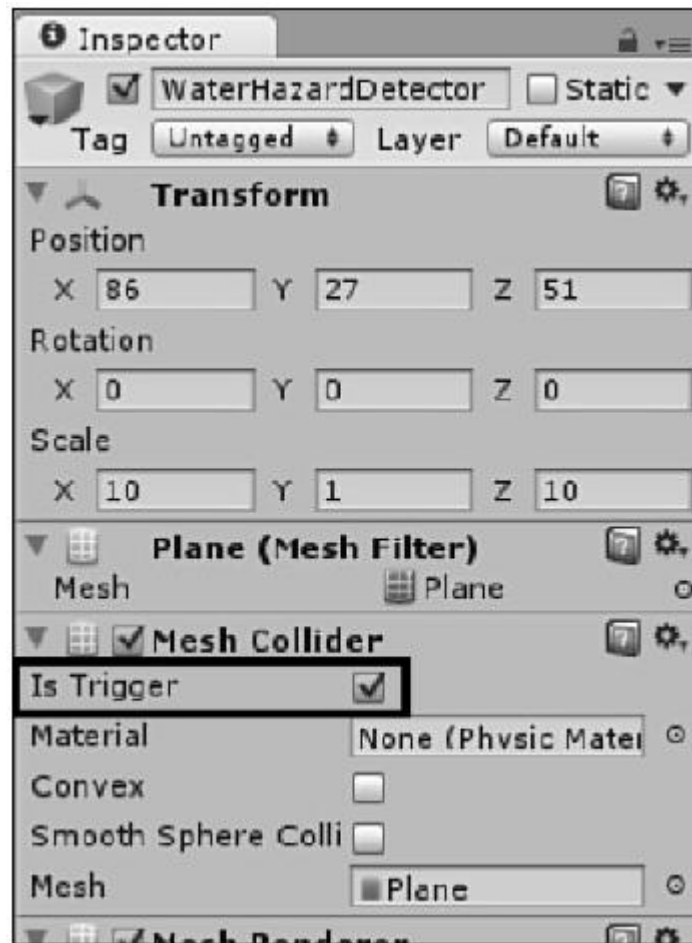


图 1.3 WaterHazard Detector 对象的 Inspector 视图

接下来，你希望给游戏添加完成区域。这个区域将是一个简单的对象，它上面带有一个点光源，使得玩家知道要去往哪里。该对象连接有一个胶囊碰撞器（capsule collider），以便它了解玩家何时可以进入该区域。要添加完成区域对象，可以遵循下面这些步骤。

- （1）向场景中添中一个空的游戏对象，并把它定位于(26, 32, 24)处。
- （2）在 Hierarchy 视图中把该对象重命名为 Finish。
- （3）给完成区域对象添加一个灯光组件（选取该对象，单击 Component > Rendering > Light 命令）。如果它的类型还不是 Point，就把类型更改为 Point，并把范围和强度分别设置为 35 和 3。
- （4）选取完成区域对象并单击 Component > Physics > Capsule Collider 命令，给该对象添加一个胶囊碰撞器。在 Inspector 视图中把 Radius 属性改为 9，并选中 IsTrigger 复选框，如图 1.4 所示。

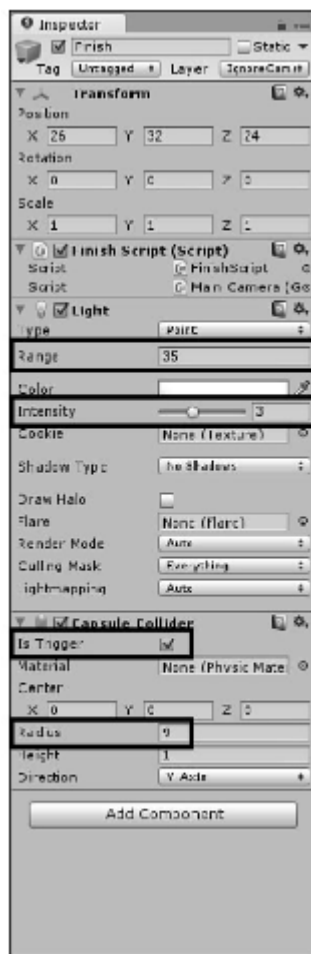


图 1.4 Finish 对象的 Inspector 视图

需要创建的最后一个对象是游戏控制对象。这个对象从技术上讲不需要存在，可以代之以只把它的属性应用于游戏世界里的其他一些持久的对象，比如 **Main Camera**。不过，你一般都会创建它自己的对象，以防止意外的删除。在开发的这个阶段，游戏控制对象是非常基本的，以后将更多地使用它。要创建游戏控制对象，可以遵循下面这些步骤。

- (1) 向场景中添加一个空的游戏对象。
- (2) 在 **Hierarchy** 视图中把该游戏对象重命名为 **GameControl**。

1.3.2 添加脚本

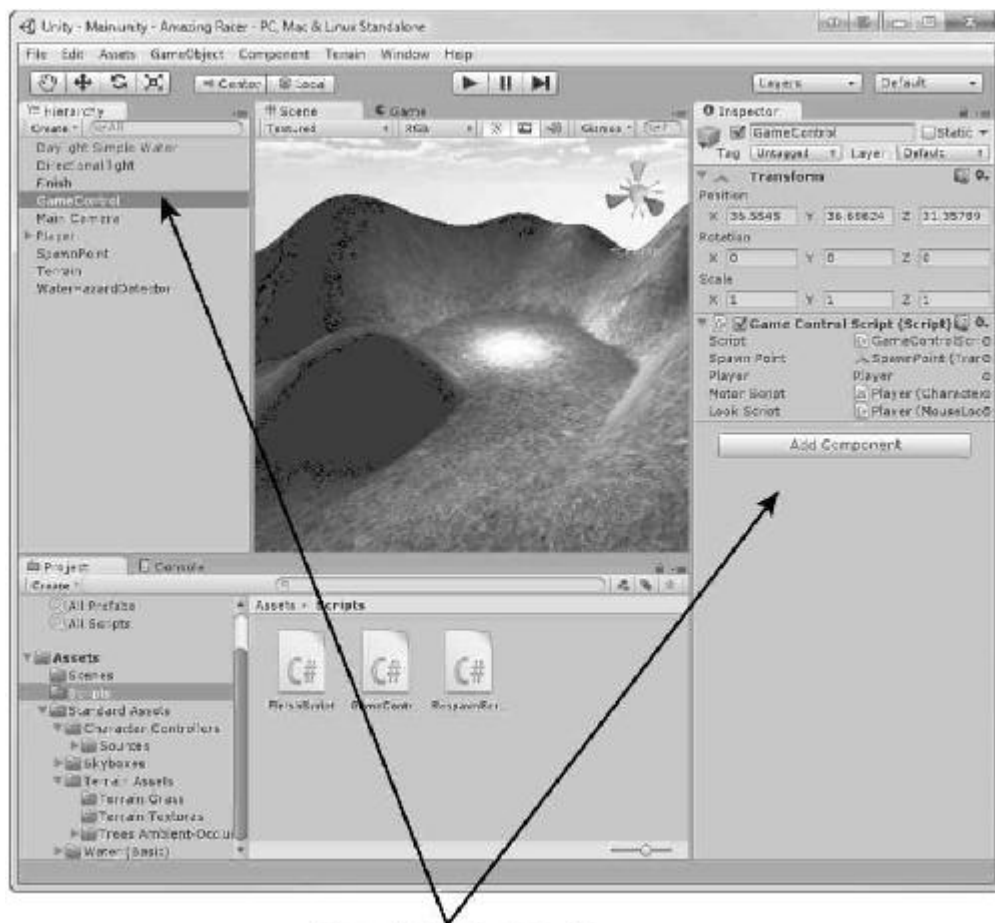
如前所述，脚本指定了游戏对象的行为。在本节中，将对游戏对象应用脚本。此时，理解这些脚本用于做什么对你来说并不重要。你需要做的第一件事是把脚本添加到项目中。

- (1) 在 **Project** 视图中的 **Assets** 下面创建一个 **Scripts** 文件夹。
 - (2) 在用于本实验配套资源中找到 **Scripts** 文件夹。
 - (3) 从本实验配套资源的 **Scripts** 文件夹中单击并把脚本拖到 **Unity** 中的 **Scripts** 文件夹中。
- 应该有 3 个脚本：**FinishScript**、**GameControlScript** 和 **RespawnScript**。

一旦脚本位于项目中，应用它们就很容易。要应用一个脚本，只需把它从 **Project** 视图中拖到你应用它的任何对象上即可，如图 1.5 所示。应用下面这些脚本。

- 对 **Finish** 游戏对象应用 **FinishScript**。
- 对 **GameControl** 对象应用 **GameControlScript**。

- 对 WaterHazardDetector 对象应用 RespawnScript。



任何一种方式都可以工作

图 1.5 通过把脚本拖到游戏对象上来应用它们

1.3.3 把脚本连接在一起

如果通读脚本，就会注意到它们都具有用于其他对象的占位符。这些占位符允许一个脚本与另一个脚本交流。你会看到，对于脚本中存在的每个占位符，在 **Inspector** 视图中用于那个脚本的组件中都会有一个属性。就像脚本一样，通过单击和拖动把对象应用于占位符，如图 1.6 所示。

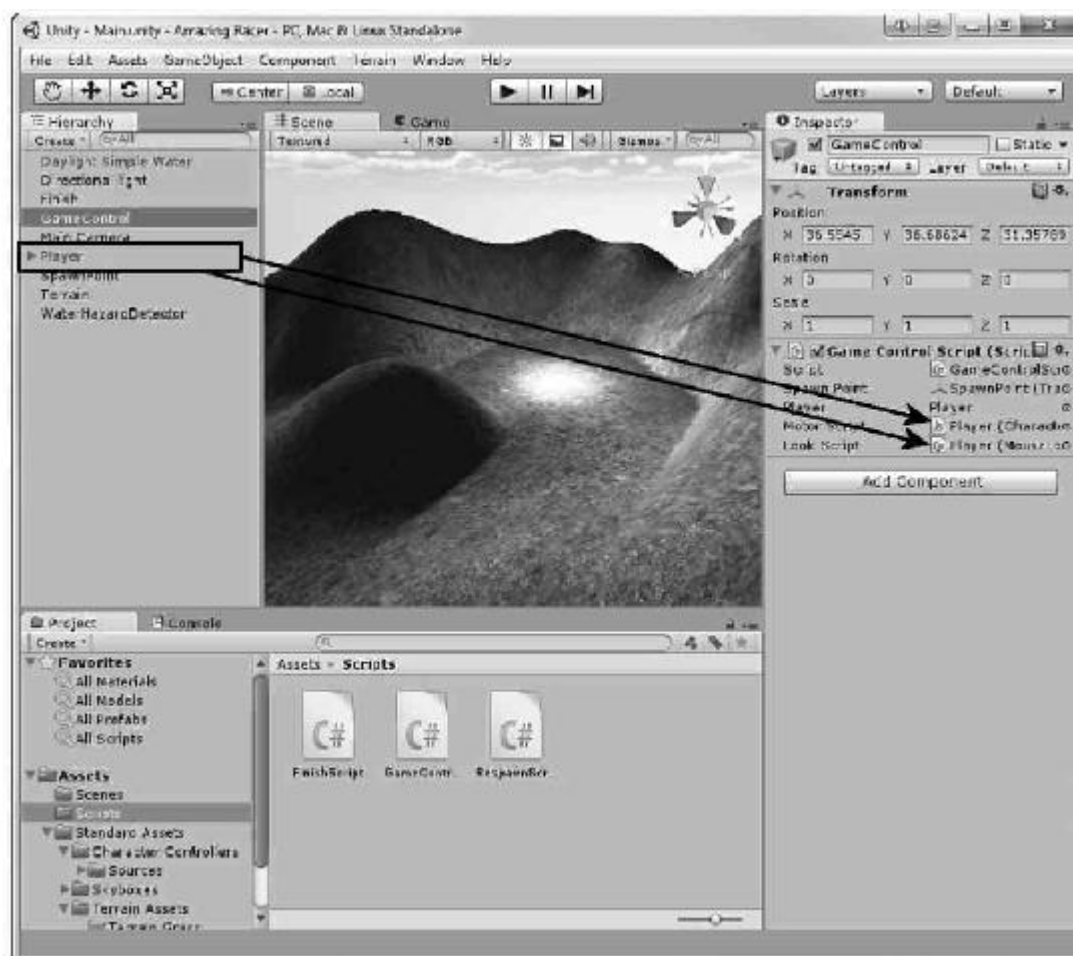


图 1.6 把游戏对象移到占位符上

首先，开始把对象与 **WaterHazardDetector** 连接起来。在 **Hierarchy** 视图中选取 **WaterHazardDetector**，并且注意到它怎样具有 **Respawn Script** 组件。这是在上一节中应用复活脚本的结果。你还注意到复活组件具有一个 **Respawn Point** 属性，这个属性是用于在前面创建的 **SpawnPoint** 游戏对象的占位符。在选取了 **WaterHazardDetector** 对象之后，从 **Hierarchy** 视图中单击并把 **SpawnPoint** 对象拖到 **Respawn Script** 组件的 **Respawn Point** 属性之上。现在，无论何时玩家陷入水障中，都将把他们移回关卡开始位置的复活点处。

要设置的下一个对象是 **Finish** 游戏对象。选取 **Finish** 游戏对象，从 **Hierarchy** 视图中单击并把 **GameControl** 对象拖到 **Inspector** 视图中的 **Finish Script** 组件的 **Game Control Script** 属性上。现在，无论何时玩家进入完成区域，游戏控制都将会得到通知。

需要设置的最后一个对象是 **GameControl**。要正确地设置这个控制，可以遵循下面这些步骤。

- (1) 单击并把 **SpawnPoint** 对象拖到 **GameControl** 的 **Game Control Script** 组件的 **Spawn Point** 属性上。
- (2) 单击并把 **Player** 对象（这是角色控制器）拖到 **GameControl** 的 **Game Control Script** 的 **Player** 属性、**Motor Script** 属性和 **Look Script** 属性上。
- (3) 需要在 **Player** 角色控制器上禁用摄像机的鼠标状态脚本。为此，可以在 **Hierarchy** 视图中展开 **Player** 对象（单击 **Player** 左边的箭头以展开它），然后选择嵌套在 **Player** 下面的 **Main Camera**。在 **Inspector** 视图中定位 **Mouse Look (Script)** 组件，并取消选中它，如图 1.7 所

示。

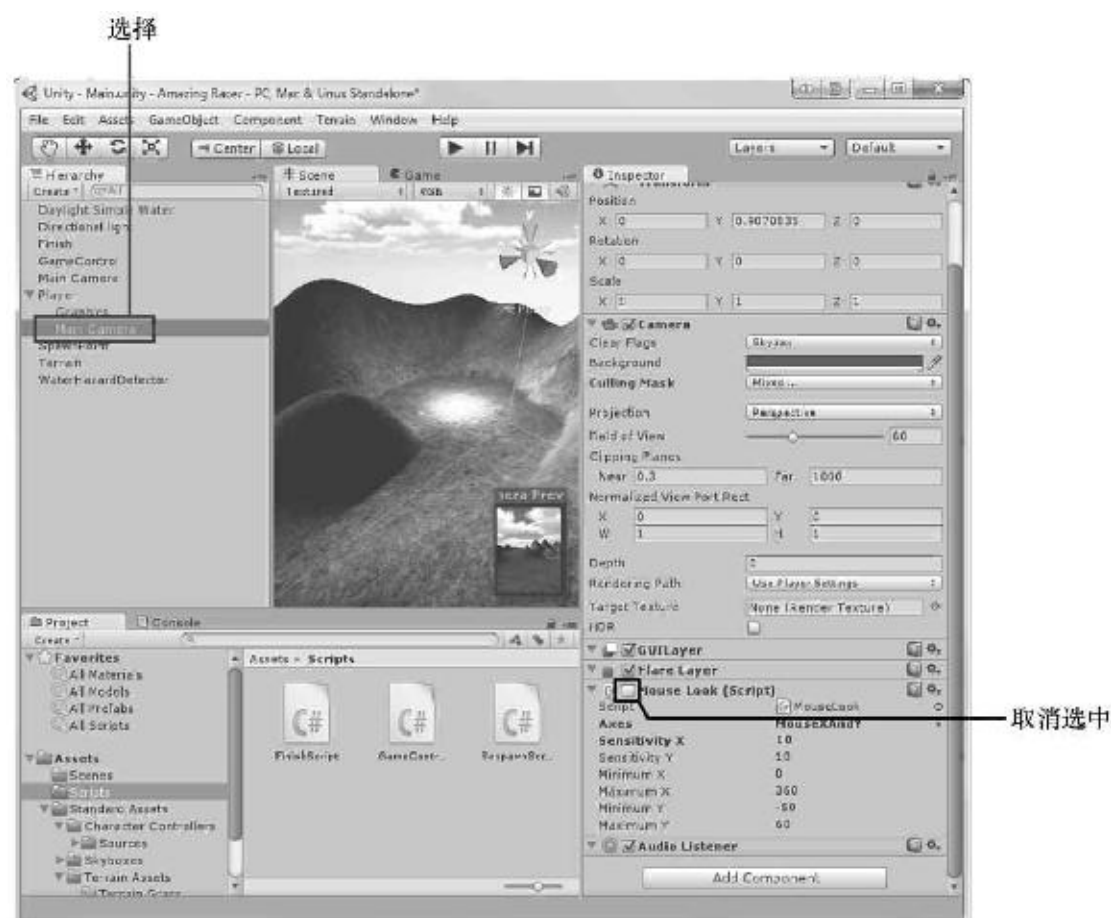


图 1.7 在玩家的摄像机上取消选中 Mouse Look 组件

这样就把游戏对象都连接好了。游戏现在完全具有可玩性！游戏中的一些部分目前可能还没有意义，但是随着你对它了解得越多，它将变得越直观。

1.4 游戏测试

你的游戏现在就完成了，但是还不能就此止步。现在，你必须开始游戏测试的过程。游戏测试是指带着找出错误或者不如你所愿的事情的意图来玩游戏。在很多时候让其他人测试你的游戏可能是有益的，以便他们可以告诉你什么对于他们是有意义的，以及他们发现什么是令人愉悦的。

如果你遵循前面描述的所有步骤，应该不会找出任何错误（通常称为 bug）。不过，确定哪些部分是有趣的过程完全取决于游戏制作者的意见。因此，这个部分留给你完成。可以玩一玩游戏，并看看你不喜欢什么。要重点注意那些不让你感到愉悦的事情。不过，不要只关注负面的东西，还要发现你所喜欢的事情。你更改这些事情的能力可能目前会受到限制，因此要把它们记下来。如果给你机会，就要规划如何把游戏改变得更好。

目前可以进行调整以使游戏变得更令人愉悦的一件简单的事情是玩家的速度。如果玩过两次这款游戏，就可能会注意到角色移动得太慢，并且这可能使人感到游戏很长并且拖拖拉拉。为了使角色变得快一些，需要修改 Player 对象上的 Character Motor (Script)组件。在 Inspector 视图中展开 Movement 属性，并且修改最大前进速度，如图 1.8 所示。示例项目把

它设置为 12。试试这个设置，看看你是否喜欢它。试试更快或更慢的速度，并选择一个你喜欢的速度。

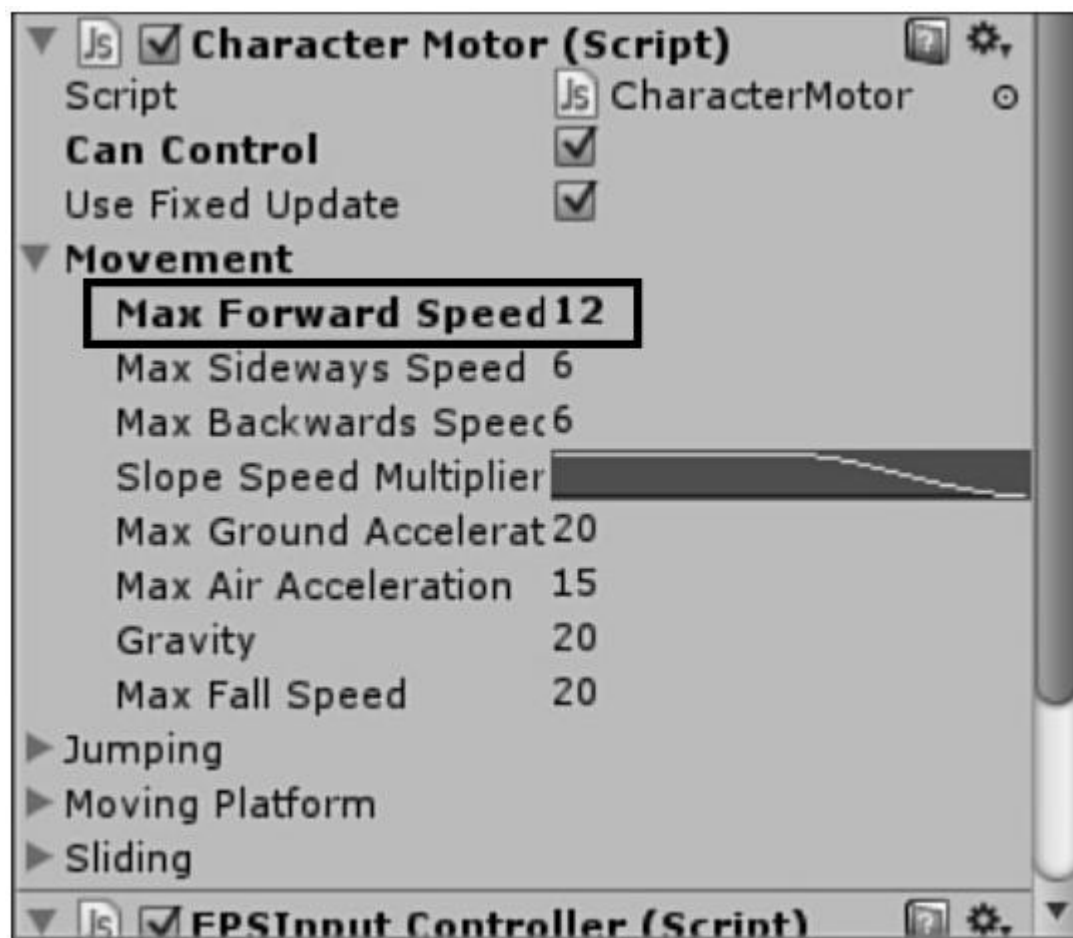


图 1.8 更改玩家的速度

1.5 小结

在本实验中，你在 Unity 中制作了自己的第一款游戏。你首先设计了游戏理念、规则和需求的多个方面。接着，你构建了游戏世界，并且添加了环境效果。然后，你添加了交互性所需的游戏对象。你对那些游戏对象应用了脚本，并把它们联系在一起。最后，你对游戏进行了测试，并且注明了你喜欢和不喜欢的东西。

1.6 问与答

问：这似乎超越了我的理解力，我做错了什么事情吗？

答：根本没有！这个过程对于不习惯它的人可能感到非常生疏。保持阅读和学习书中的材料，慢慢就会开始熟悉这个过程。你能够做的最好的事情是注意对象如何通过脚本彼此产生联系。

问：你没有介绍如何构建和部署游戏。为什么？

答：构建和部署游戏在教材中有专门章节加以介绍。在构建游戏时有许多事情要考虑，此时，你只应该把注意力集中在开发它所需的理念上。

问：如果没有脚本，为什么我们不能制作游戏？

答：如前所述，脚本定义了对象的行为。如果没有某种形式的交互式行为，将很难具有一款条理分明的游戏。在第 8 章和第 9 章学习编写脚本之前，就在第 7 章中构建一款游戏的唯一原因是：应该在转向不同的内容之前强化你已经学过的主题。

1.7 测验

花一些时间做完这里的问题，确保你完全掌握了本实验所学的知识。

1.7.1 问题

- 1.什么是游戏的需求？
- 2.什么是这款游戏的获胜条件？
- 3.在这种地形中，建议使用多少种纹理以获得一种自然混合的外观？
- 4.哪个对象负责控制游戏的流程？
- 5.为什么我们要测试游戏？

1.7.2 答案

- 1.需求是制作游戏需要创建的资源列表。
- 2.这是一个有意捉弄人的问题！这款游戏没有明确的获胜条件。当玩家这一次到达的时间比上一次更短时，就认为他或她获胜了。不过，没有以任何方式把它构建到游戏中。
- 3.3 种：青草、青草和岩石、岩石。
- 4.游戏控制器。在这款游戏中，它被命名为 **GameControl**。
- 5.要发现错误，以及确定游戏的哪些部分像我们希望的那样工作。

1.7.3 练习

对于制作游戏，最令人愉快的体验便是可以使它们像你所想的那样运行。遵循指导可能是一种良好的学习经历，但是将不能获得制作一款自定义游戏的满足感。在这个练习中，你将有机会稍微修改一下游戏，以使事物更独特。至于你想如何修改游戏，则完全取决于你自己。下面列出了一些建议。

- 尝试添加多个完成区域。看看你在放置它们之后，是否可以给玩家提供更多的选择。
- 修改地形，使之具有更多的或不同的障碍。只要像水障那样构建这些障碍（包括脚本），它们就会工作得很好。
- 尝试具有多个复活位置，并且使其中一些障碍把你移到第二个或第三个复活点。
- 修改天空和纹理，创建一个陌生的游戏世界。并且提供独特的游戏世界体验。